

## Анализ диаграммы классов системы учета дефектов

### 1. Проблемы соответствия требованиям

#### 1.1. Отчеты

- Отсутствует класс отчета для бригадира в конце каждой смены  
**Решение:** Добавить класс BrigadierShiftReport с полями shiftDate, workerStatistics: Map < WorkerId, WorkerStatistics >
- Нет класса-менеджера для создания, отправки QualityControlReport, AccountingReport и др.  
**Решение:** Добавить класс ReportManager с методами для генерации и отправки отчетов
- В ShiftReport для начальника цеха избыточная информация repairDetails - эти данные нужны в QualityControlReport
- В QualityControlReport не указан участок конвейера, к которому относятся данные в отчете
- В AccountingReport поле workerShiftDetails: List<string> не информативно - нужно хранить данные о количестве смен каждого работника.  
**Решение:** workerShiftDetails: Map<int, int>

#### 1.2. Функционал Dispatcher

- Не хватает методов, отвечающих за:
  - Зафиксировать время начала и конца ремонта,
  - Зафиксировать рабочего, выполнявшего ремонт и ремонтное место, занимаемое автомобилем в процессе ремонта
  - Видеть свободные и занятые места в ремонтных зонах своего участка конвейера, а также наличие свободных рабочих в зоне
- Лишний метод monitorRepairStatus  
**Решение:** добавить нужные методы и удалить лишние.

#### 1.3. Управление производством

- ProductionPlan есть, но нет метода для определения порядка запуска производства  
**Решение:** добавить метод generateProductionOrder(): List<Order>
- Нет методов для:
  - Сообщения о дефектах (должен быть у Worker или Dispatcher).
  - Внесения данных о рабочих зонах и местах, бригадирах и бригадах.
  - У Worker нет метода для регистрации себя как доступного для работы
  - У бригадира нет метода для назначения ремонта рабочему

**Решение:** добавить Worker методы reportDefect, setAvailability для рабочих, а для бригадира - метод assignRepairToWorker, добавить методы Dispatcher для внесения данных о рабочих зонах и местах, бригадирах и бригадах.

#### 1.4. Учет рабочих

- Класс Worker имеет атрибут available, но нет информации о сменах, графиках работы или учете отработанного времени, что важно для отчетов  
**Решение:** добавить класс Shift (shiftId, startTime, endTime, workerId) и связать его с Worker. Это упростит учет отработанных смен и генерацию отчетов.

- в классе RepairTeam поле teamWorkers лучше сделать List<int> с workerId, а в Worker добавить поле = id бригады, с которой он работает. Это упростит получение информации о работнике. И тогда в классе Repair можно убрать поле repairTeam и не хранить избыточную информацию.

## 2. Проблемы согласованности, избыточности и формата данных

- Все классы имеют id, но связи реализованы через объекты, а не id - надо унифицировать - либо везде использовать id, либо объекты.  
**Решение:** везде использовать Id, добавить методы для получения объекта по Id - для снижения связанности, эффективности памяти и удобства.
- в AccountingReport логичнее было бы сделать month типа Month или String для полной записи даты, а не просто int для номера месяца.  
**Решение:** month типа Month
- В поля Defect также надо добавить поле repairId для понимания на каком ремонте исправляется дефект. А в классе Car поле repairHistory по требованиям избыточно, на список ремонтов можно будет посмотреть через список дефектов.

## 3. Проблемы с наследованием

- Worker - единый класс для обычного рабочего и бригадира, но их функционал сильно отличается. Совпадает только name и workerId. Поле available нужно только для рабочих.  
**Решение:** разделить на два класса, добавить абстрактный класс Employee для общих полей и методов, от которого, помимо Worker и Foreman, также может extends класс Dispatcher

## 4. Проблемы с отношениями классов

- В текущей диаграмме связь между AccountingReport и Worker выглядит неочевидной - непонятно, как именно отчет получает данные о сменах работников  
**Решение:** добавить метод для получения списка смен работников и создании workerShiftDetails в отчете на его основе

## 5. Улучшение организации

- Лучше разделить классы на диаграмме на логические модули: "Ремонт", "Персонал", "Отчеты" и т.д.  
**Решение:**  
Ремонт - Car, Defect, Repair, RepairZone, RepairPlace;  
Персонал - Dispatcher, Worker, Foreman, Shift, RepairTeam;  
Отчеты - ShiftReport, QualityControlReport, AccountingReport, ForemanShiftReport;  
Производство - ProductionPlan, Order.